Imam Abdulrahman Bin Faisal University

College of Computer Science & Information Technology
Programming for AI – ARTI 303/402

SENTIMENT ANALYZER DASHBOARD FOR AMAZON REVIEWS

Course Instructor: Mohammad Aftab Khan

Team Members

Wesam Jaber Mohammed Almalki — 2240005660
Fawaz Mohammed Saeed Alshahrani — 2240005571
Jaber Saleh Mohammed Alsaykhan — 2240006539
Mohammed Talal Hussain Altuwairqi — 2240005295
Anas Yahya Hamzi — 2240003869

Table of Contents

Abstract *4*

1.Introduction *5*

2.Problem Statement *5*

3. Project Objectives *6*

4.Project Scope *6*

5.Dataset Description *7*

6.System Requirements *8*

Functional Requirements (FR) **8**

7.System Design *10*

System Architecture *10*

Flowcharts (Description) *11*

Data Loading Flowchart (Description) *11*

Process Steps: 11

Sentiment Analysis Flowchart (Description) *12*

Process Steps: 12

Product Analysis Flowchart (Description) *13*

Process Steps: 13

Visualization Flowchart (Description) *14*

Process Steps: 14

Model Training Flowchart (Description) *15*

Process Steps: 15

Class Diagram (Description) *16*

Main Classes 16

1. DataLoader **16**

2. SentimentModel **17**

3. MLModel **18**

..... **18**

4. ProductAnalyzer **19**

5. AIRecommender **20**

6. ChartBuilder **20**

..... **21**

7. SentimentAppGUI (Main GUI Class) **23**

Relationships Between Classes 23

8.Implementation *25*
Programming Language 25
Development Environment 25
Libraries Used *25*
System Modules and Their Implementations *25*
1. DataLoader Module 25
2. SentimentModel Module 26
3. MLModel Module 26
4. ProductAnalyzer Module 26
5. ChartBuilder Module 26
6. SentimentAppGUI Module 26
System Workflow Summary *27*
Code Structure (Folder Layout) *27*
9.Testing *28*
12.1 Testing Objectives **28**
12.2 Test Environment **28**
12.3 Test Cases **28**
Test Case 1: Dataset Loading 28
Test Case 2: Data Loading with Invalid File 29
Test Case 3: Sentiment Prediction 29
Test Case 4: Model Training 29
Test Case 5: Visualization Generation 29
Test Case 6: GUI Interaction 30
10.Conclusion *30*
References *31*
Appendix *32*
Appendix A — Source Code Files **32**
A.1 analyzer.py 32
A.2 ML.py 32
A.3 charts.py 33
A.5 ai_recommend.py 35
A.6 dataL.py 35
Appendix B — Dataset Sample **36**

Table of figures ant table

Figure 1: CSV File Loading Process Flowchart 11
Figure 2 : Sentiment Prediction Pipeline 12
Figure 3: Sentiment Aggregation and Statistics Generation 13
Figure 4: Bar Chart Generation Workflow 14
Figure 5: Model Training Workflow 15
Figure 6:Figure X: File Loading Methods in the DataLoader Class 16
Figure 7 :Sentiment Analysis Functions in the Model Class 17
Figure 8: Sentiment Analysis Model Class Functions 18
Figure 9 : Utility Functions for Data Analysis and Feature Extraction 19
Figure 10 :Sentiment Summary Generation Function 20
Figure 11 : ChartBuilder Visualization Methods 20
Figure 12 : Sentiment Distribution Bar Chart 21
Figure 13 : Sentiment Distribution Pie Chart 21
Figure 14 :Rating vs. Polarity Scatter Plot 22
Figure 15 : Polarity Histogram 22
Figure 16: Sentiment Polarity Over Index (Line Chart) 22
Figure 17 : Application Initialization and GUI Setup Code 23
Figure 18 : Sentiment Analyzer Dashboard Interface 24

Table 1: Libraries Used and Their Purposes 25
Table 2: Project Directory Structure and Class Package Files 27

Abstract

This project presents the development of an AI-powered Sentiment Analyzer Dashboard designed to automatically classify Amazon mobile-phone reviews into positive or negative sentiments using Natural Language Processing (NLP) and machine learning techniques. The system processes a large dataset of more than 400,000 customer reviews and extracts meaningful insights related to product satisfaction and user experience. It integrates multiple components including data loading, text preprocessing, TF-IDF vectorization, logistic regression–based sentiment classification, product-level analysis, and interactive data visualization through a Tkinter-based GUI. The results demonstrate the system's ability to efficiently analyze large-scale review data and provide decision-support insights useful for product evaluation and customer-experience improvement.

1.Introduction

The rapid growth of online shopping platforms such as Amazon has resulted in an overwhelming number of customer reviews for each product. These reviews represent a valuable source of feedback for consumers, manufacturers, and sellers, but the sheer volume makes manual analysis inefficient and impractical. Sentiment analysis, a field within Natural Language Processing (NLP), enables the automated extraction of opinions and emotions from text data, making it possible to transform large collections of unstructured reviews into meaningful insights.

This project introduces a Sentiment Analyzer Dashboard designed by (gui) to analyze Amazon reviews of unlocked mobile phones. The system leverages machine learning techniques to classify reviews as positive or negative and provides visual summaries that support better understanding of customer satisfaction trends. In addition to sentiment classification, the system offers product-level insights, statistical summaries, and an interactive graphical interface that allows users to load datasets, perform sentiment analysis, and visualize results in real time.

The dashboard serves as a practical demonstration of integrating data processing, machine learning, visualization, and user interface components into a unified application, offering both educational value and real-world applicability in fields such as marketing, product analysis, and customer experience research.

2.Problem Statement

Online marketplaces such as Amazon receive an enormous volume of customer reviews for each product, making it difficult for consumers and businesses to interpret customer opinions accurately. Manually reading thousands of reviews is time-consuming, inconsistent, and prone to bias. Without an efficient method for analyzing this unstructured text data, valuable insights regarding customer satisfaction, product quality, and user experience remain hidden. Therefore, there is a need for an automated system capable of processing large-scale review datasets and extracting sentiment patterns that can support decision-making for customers, sellers, and manufacturers.

3. Project Objectives

The main objectives of this project are as follows:

1. To develop an automated system capable of analyzing large volumes of Amazon product reviews.
2. To classify customer reviews into positive or negative sentiments using machine learning techniques.
3. To preprocess and transform raw text data into meaningful features using TF-IDF vectorization.
4. To build a visualization dashboard that displays sentiment distributions and product-level insights.
5. To provide an interactive and user-friendly GUI that allows users to load datasets and generate sentiment analysis results in real time.
6. To support decision-making for customers, sellers, and product manufacturers by summarizing key sentiment trends.

4.Project Scope

In scope

The scope of this project focuses on designing and implementing a sentiment analysis system capable of processing Amazon reviews for unlocked mobile phones. The system includes data loading, text preprocessing, sentiment classification using machine learning, product-level review analysis, and visualization through an interactive dashboard. A Tkinter-based graphical user interface (GUI) is provided to allow users to upload datasets, execute sentiment analysis, and view generated results and charts.

Out of scope

The project does not include real-time data collection from online platforms, multilingual sentiment processing, deep-learning–based classification models, or deployment as a web or mobile application. Additionally, the system is limited to binary sentiment classification (positive/negative) and does not perform aspect-based sentiment analysis or detect emotions beyond basic polarity.

5.Dataset Description

The dataset used in this project consists of approximately 400,000 Amazon reviews of unlocked mobile phones, collected and published by PromptCloud. Each record in the dataset contains multiple attributes that describe customer feedback and product characteristics. The primary fields include:

- **Review Text:** The written review provided by the customer.
- **Rating:** A numerical rating typically ranging from 1 to 5 stars.
- **Product Name:** The specific mobile device associated with the review.
- **Review Date:** The date on which the review was submitted.
- **Additional Metadata:** Such as reviewer ID, product ID, and helpfulness metrics.

For the purpose of this project, the most important feature is the **textual review**, which is processed to determine whether the sentiment expressed is positive or negative. The dataset provides a sufficiently large and diverse sample of consumer opinions, making it suitable for training and evaluating machine learning models for sentiment classification.

6.System Requirements

Functional Requirements (FR)

1. **Data Loading:**
   The system shall allow users to upload review datasets in CSV format through the GUI.
2. **Text Preprocessing:**
   The system shall clean and prepare review text by removing missing values, converting text to lowercase, and applying tokenization and vectorization.
3. **Sentiment Classification:**
   The system shall classify each review as positive or negative using a trained machine learning model.
4. **Model Training:**
   The system shall allow offline training of a logistic regression model using TF-IDF features.
5. **Product-Level Analysis:**
   The system shall generate summary statistics for individual products such as sentiment counts and distribution.
6. **Visualization:**
   The system shall display charts including sentiment distribution bar charts and other visual summaries.
7. **Dashboard Interaction:**
   The GUI shall provide interactive buttons for loading data, running sentiment analysis, and displaying results.
8. **Recommendation Output:**
   The system shall provide basic insights or recommendations based on sentiment patterns.

Non-Functional Requirements (NFR)

1. **Usability:**
   The GUI must be easy to use, intuitive, and responsive for users with minimal technical experience.
2. **Performance:**
   The system should process and classify large datasets efficiently without significant delays.
3. **Scalability:**
   The architecture should allow future expansion to support more features such as multi-class sentiment or product comparison.
4. **Reliability:**
   The system must operate consistently and handle missing or corrupted data gracefully.
5. **Maintainability:**
   The code should be modular and structured to allow future updates, model retraining, and feature extension.
6. **Compatibility:**
   The system shall run on standard Python environments and be compatible with major operating systems.
7. **Security:**
   The system should not expose user data and must handle local files securely without external transmission.

7.System Design

System Architecture

The overall architecture of the Sentiment Analyzer Dashboard is designed using a modular, multi-layered structure that separates data processing, machine learning, visualization, and user interaction. This modular approach enhances maintainability, scalability, and clarity of the system. The architecture consists of the following main layers and components:

1. **Data Loading Layer**

   This component is responsible for importing review datasets from CSV files. It handles file selection, reading, and initial validation. The DataLoader class manages these operations and ensures that the dataset is properly structured before processing.

2. **Preprocessing and Feature Extraction Layer**

   After loading the data, the system preprocesses the textual reviews by cleaning the text and converting it into numerical features using TF-IDF vectorization. This step transforms raw text into machine-readable form suitable for training and prediction.

3. **Machine Learning Layer**

   This layer uses a Logistic Regression classifier to perform binary sentiment analysis. The trained model outputs sentiment predictions (positive or negative) based on the extracted TF-IDF features. The SentimentModel and MLModel components perform training, testing, and prediction.

4. **Analysis and Recommendation Layer**

   This component aggregates model results to generate product-level statistics. It calculates sentiment distributions and supports insight generation based on customer feedback patterns. The ProductAnalyzer and AIRecommender classes belong to this layer.

5. **Visualization Layer**

   Visual outputs such as bar charts and sentiment distribution graphs are generated through the ChartBuilder component. This layer converts numerical results into interpretable visualizations.

6. **Graphical User Interface (GUI) Layer**

   Built using Tkinter, the GUI integrates all system components. Users can load datasets, trigger sentiment analysis, train models, and view results through interactive buttons and display areas.

Together, these layers create a complete pipeline that flows from data loading to analysis, visualization, and user interaction.

Flowcharts (Description)

The flowcharts illustrate the sequential flow of operations performed by the Sentiment Analyzer Dashboard. These diagrams provide a clear overview of how data moves through the system, from loading the dataset to generating the final sentiment analysis results.

Data Loading Flowchart (Description)

Process Steps:

1. Start
2. User selects a CSV file
3. System reads the file
4. Validate file structure and contents
   - If valid → proceed
   - If invalid → show error message
5. Return loaded dataset to GUI
6. End

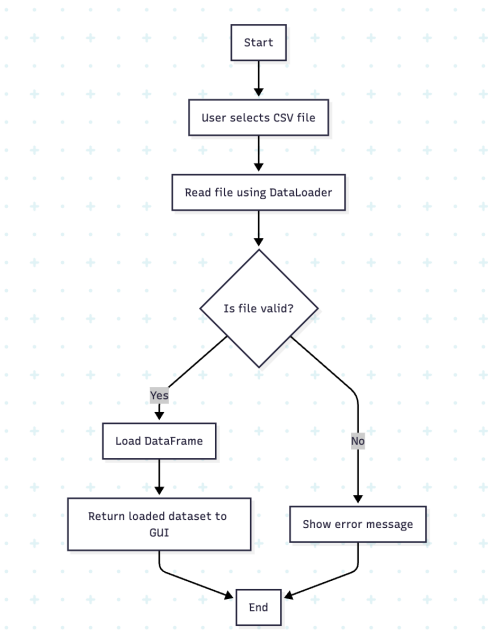

Figure 1: CSV File Loading Process Flowchart

Sentiment Analysis Flowchart (Description)

Process Steps:

1. Start
2. Receive dataset
3. Preprocess text (cleaning, handling missing values)
4. Apply TF-IDF vectorization
5. Pass data to Logistic Regression classifier
6. Predict sentiment labels
7. Return sentiment results
8. End

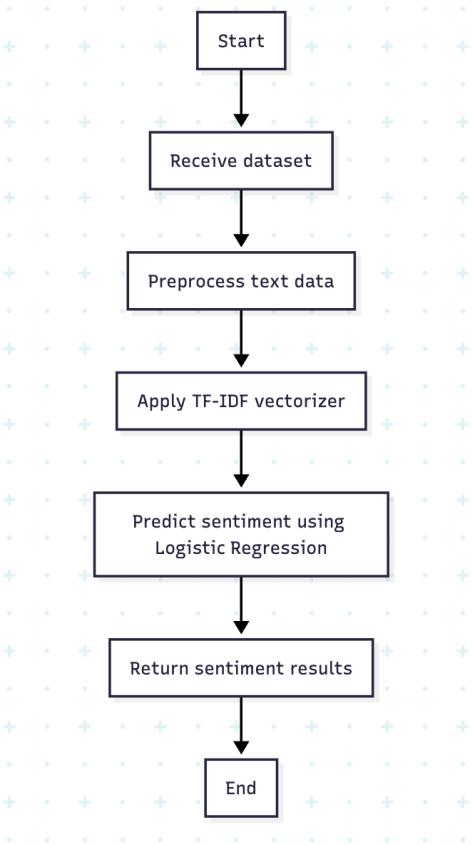

Figure 2 : Sentiment Prediction Pipeline

Product Analysis Flowchart (Description)

Process Steps:

1. Start
2. Receive sentiment predictions
3. Group results by product
4. Count positive and negative sentiments
5. Generate summary statistics
6. Output results to GUI
7. End
8.

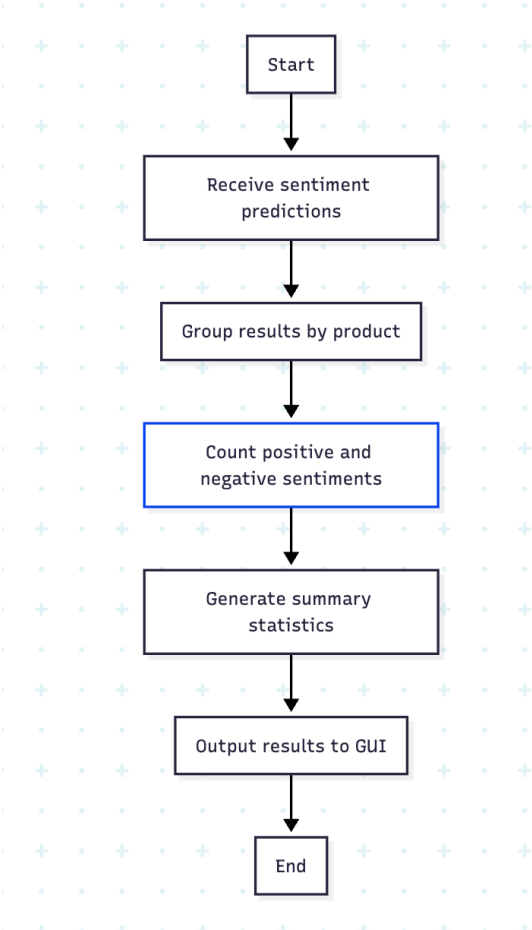

Figure 3: Sentiment Aggregation and Statistics Generation

Visualization Flowchart (Description)

Process Steps:

1. Start
2. Receive sentiment summary data
3. Generate bar chart using ChartBuilder
4. Format chart for display
5. Send chart to GUI
6. GUI displays the chart
7. End

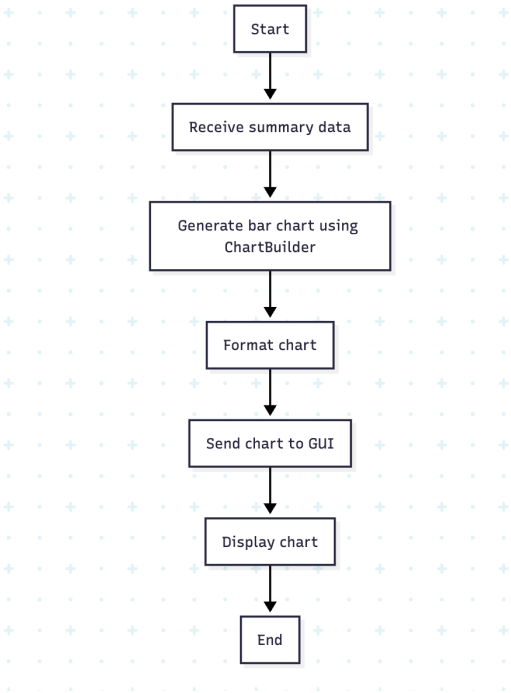

Figure 4: Bar Chart Generation Workflow

Model Training Flowchart (Description)

Process Steps:

1. Start
2. Load training data
3. Preprocess text
4. Apply TF-IDF vectorization
5. Train Logistic Regression model
6. Evaluate model accuracy
7. Save trained model
8. Display accuracy to user
9. End

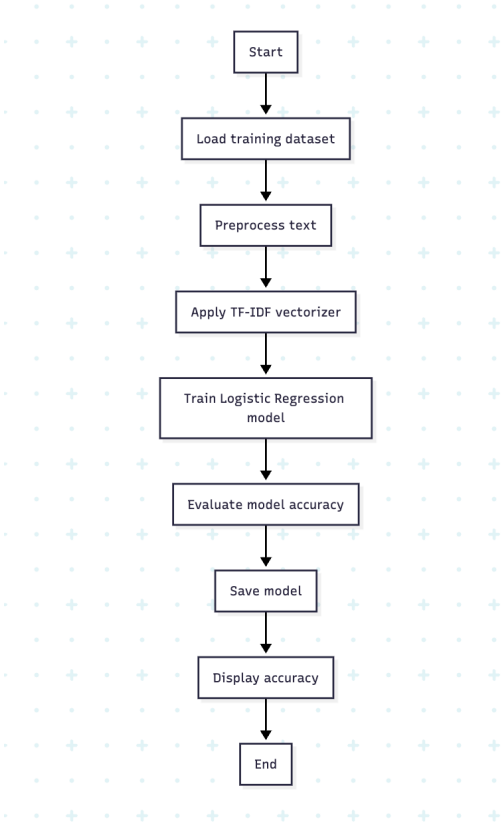

Figure 5: Model Training Workflow

Class Diagram (Description)

The Class Diagram represents the structural design of the system by defining the main classes, their attributes, methods, and relationships. The Sentiment Analyzer Dashboard is organized into modular classes to maintain clarity, scalability, and efficient separation of responsibilities.

Main Classes

1. DataLoader

Responsibilities:

- Load datasets from CSV files
- Validate file format
- Return DataFrame to GUI

Key Methods:

- load_csv(exel,zip)(filepath)
- validate_data(df)

```
def load_zip(self, path):

def load_csv(self, path):
    return pd.read_csv(path)

def load_excel(self, path):
    return pd.read_excel(path)
```

Figure 6:Figure X: File Loading Methods in the DataLoader Class

2. SentimentModel

Responsibilities:

- Provide sentiment prediction functionality using a trained model
- Preprocess text for predictions

Key Methods:

- anla(text)
- load_model()
- prepare_text(text)

```
def analyze_text(self, text):
    if pd.isna(text):
        return "Neutral"

    score = TextBlob(str(text)).sentiment.polarity

    if score < -0.2:
        return "Negative"
    elif score > 0.2:
        return "Positive"
    return "Neutral"

def apply_model(self, df, review_col="Reviews"):
    df["Polarity"] = df[review_col].apply(
        lambda t: TextBlob(str(t)).sentiment.polarity
    )
    df["Sentiment"] = df[review_col].apply(self.analyze_text)
    return df
```

Figure 7 :Sentiment Analysis Functions in the Model Class

3. MLModel

Responsibilities:

- Train the Logistic Regression classifier
- Apply TF-IDF vectorization
- Evaluate model performance

Key Methods:

- train(df)
- evaluate(X_test, y_test)
- save (path)
- load()

```
def train(self, df, text_col="Reviews", label_col="Sentiment"):
    df = df[[text_col, label_col]].dropna()

def predict(self, text):
    X = self.vectorizer.transform([str(text)])
    return self.model.predict(X)[0]

def save(self, path="sentiment_model.pkl"):
    joblib.dump((self.model, self.vectorizer), path)

def load(self, path="sentiment_model.pkl"):
    self.model, self.vectorizer = joblib.load(path)
```

Figure 8: Sentiment Analysis Model Class Functions

4. ProductAnalyzer

Responsibilities:

- Analyze sentiment results for each product
- Generate statistics

Key Methods:

- filter_product (predictions)
- get_review_stats (df)
- top_products()

```
def filter_product(self, df, keyword):
    if "Product Name" not in df.columns:
        print("ERROR: 'Product Name' column not found!")
        return pd.DataFrame()

    keyword = str(keyword).strip().lower()
    df["Product Name"] = df["Product Name"].astype(str).str.lower()

    filtered = df[df["Product Name"].str.contains(keyword, na=False)]
    return filtered

def get_review_stats(self, df):
    if df.empty:
        return None

    lengths = df["Reviews"].astype(str).apply(len)
    return {
        "count": len(df),
        "columns": df.shape[1],
        "min_length": lengths.min(),
        "max_length": lengths.max(),
        "avg_length": lengths.mean()
    }

def top_products(self, df, n=10):
    if df.empty:
        return None
    return df["Product Name"].value_counts().head(n)
```

Figure 9 : Utility Functions for Data Analysis and Feature Extraction

5. AIRecommender

Responsibilities:

- Provide basic recommendations based on sentiment trends

Key Methods:

- generate (,)

```
def generate(self, df):
    pos = len(df[df["Sentiment"] == "Positive"])
    neg = len(df[df["Sentiment"] == "Negative"])
    total = len(df)
```

Figure 10 :Sentiment Summary Generation Function

6. ChartBuilder

Responsibilities:

- Generate visual charts (bar charts, distribution graphs)
- Prepare figures for display

Key Methods:

- bar_chart(data)
- product_chart(data)

```
def bar_chart(self, df):
def histogram(self, df):
def pie_chart(self, df):
def line_chart(self, df):
def box_plot(self, df):
```

Figure 11 : ChartBuilder Visualization Methods

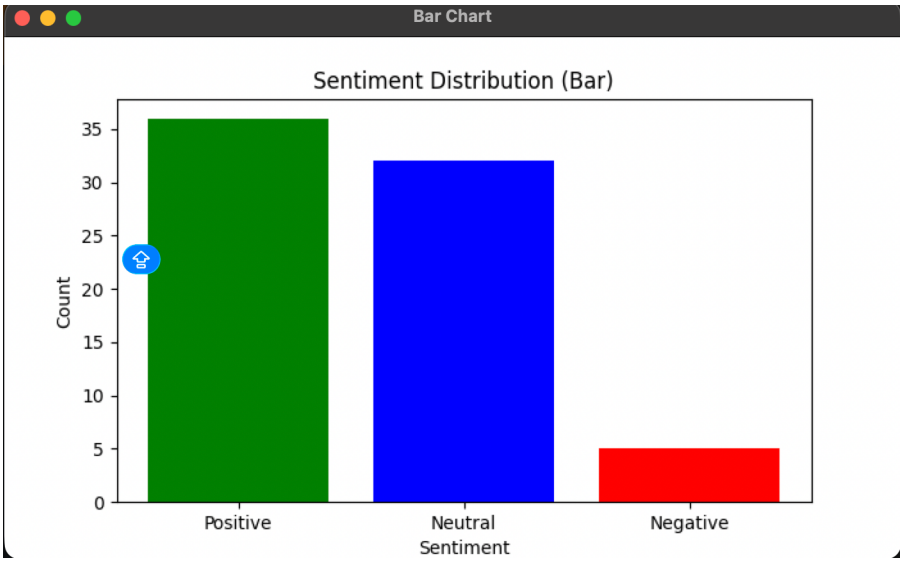

Figure 12 : Sentiment Distribution Bar Chart

This figure displays a bar chart illustrating the distribution of sentiment categories within the dataset, showing the counts of Positive, Neutral, and Negative reviews as generated by the sentiment analysis model.

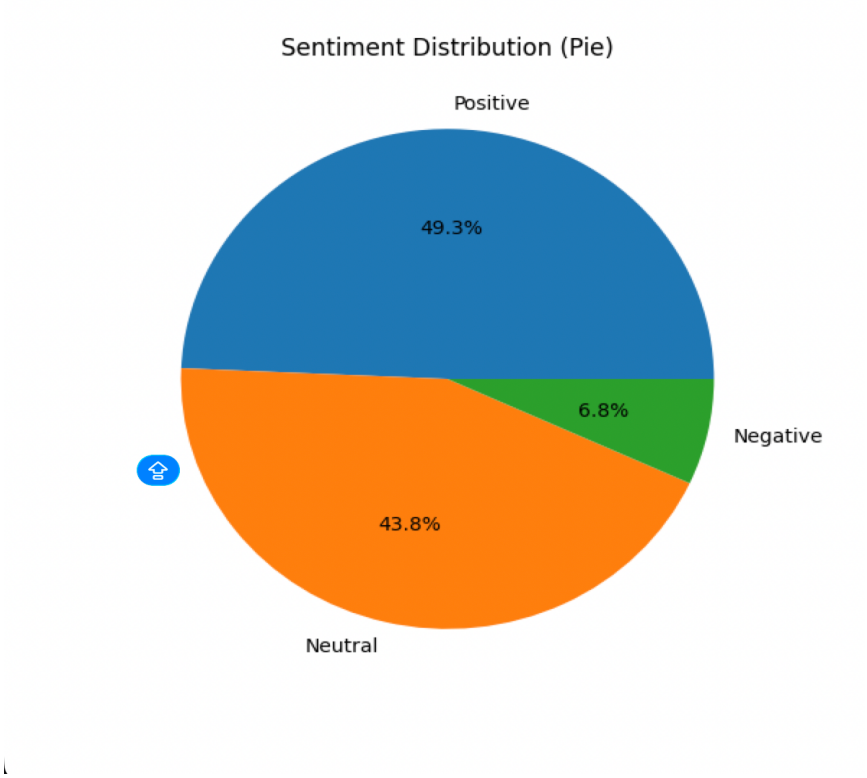

Figure 13 : Sentiment Distribution Pie Chart

the figure presents a pie chart illustrating the proportion of each sentiment category in the dataset. It visually compares the percentages of Positive, Neutral, and Negative reviews.

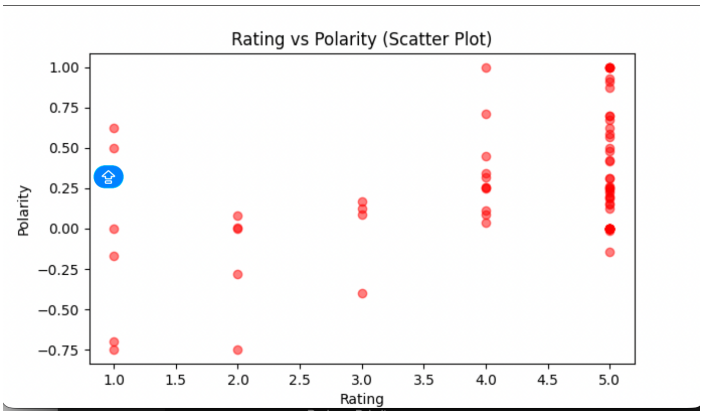

Figure 14 :Rating vs. Polarity Scatter Plot

the figure displays a scatter plot illustrating how sentiment polarity varies across different rating scores each point represents a review, allowing readers to observe trends or correlations between numerical ratings and sentiment scores.

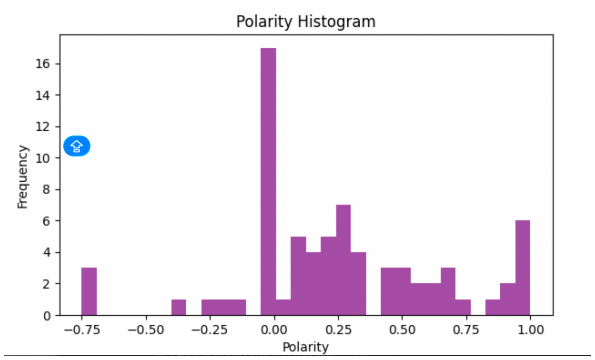

Figure 15 : Polarity Histogram

the figure presents a histogram illustrating how often different polarity values occur in the dataset this visualization helps reveal the overall spread and concentration of sentiment scores.

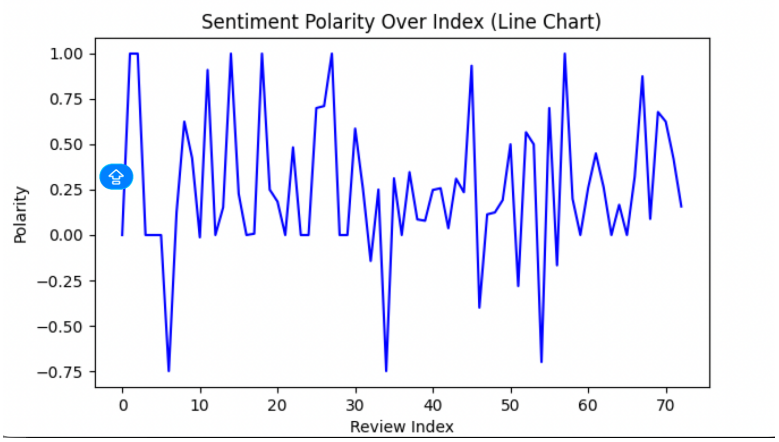

Figure 16: Sentiment Polarity Over Index (Line Chart)

the figure presents a line chart tracking the polarity values of reviews in sequence. It visualizes how sentiment fluctuates from one review to another.

7. SentimentAppGUI (Main GUI Class)

Responsibilities:

- Handle user interface logic
- Integrate all backend components
- Display results and charts

Key Methods:

- load_file()
- run_analysis()
- show_chart()
- train_model()

```
import sys
sys.path.append("./class_pkg")
```

```
from gui import SentimentAppGUI
import tkinter as tk

root = tk.Tk()
app = SentimentAppGUI(root)
root.mainloop()
```

Figure 17 : Application Initialization and GUI Setup Code

Relationships Between Classes

- **SentimentAppGUI to DataLoader:**
  The GUI calls the DataLoader to import CSV files.
- **SentimentAppGUI to MLModel:**
  Used for model training and prediction initialization.
- **SentimentAppGUI to SentimentModel:**
  Performs actual sentiment prediction on user-loaded data.
- **SentimentAppGUI to Analyzer & ChartBuilder:**
  Generates summaries and visualizations for display.
- **MLModel to SentimentModel:**

FINAL INTERFACE(GUI)

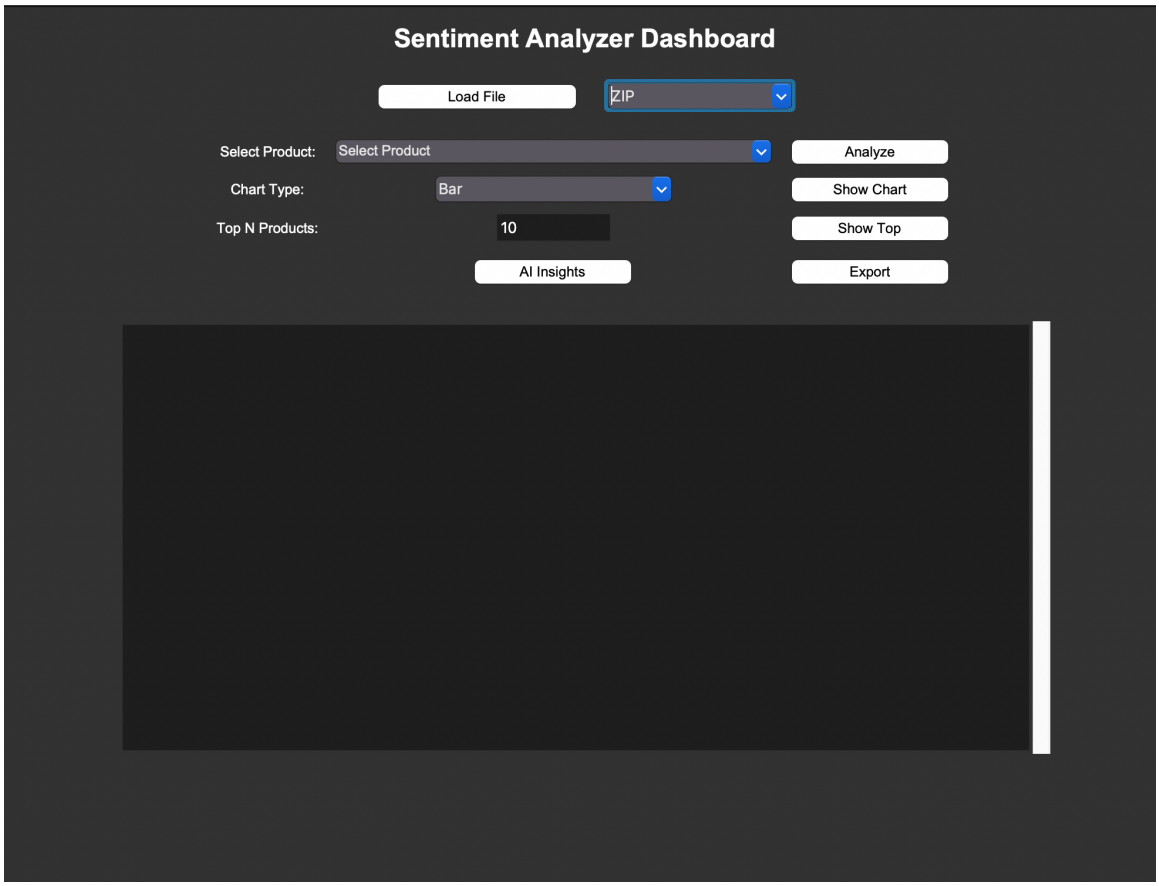

Figure 18 : Sentiment Analyzer Dashboard Interface

the figure shows the main dashboard of the application. The interface allows the user to load datasets, choose a product, run sentiment analysis, generate charts, view the top products, access AI insights, and export results The lower section of the interface is used to display visualizations and outputs.

8.Implementation

The implementation of the Sentiment Analyzer Dashboard was carried out using Python, leveraging multiple libraries for data processing, machine learning, visualization, and graphical user interface development. The system follows a modular design to ensure clarity, maintainability, and ease of future extension.

Programming Language

- The entire system was implemented in **Python 3**, chosen for its extensive machine learning, data analysis, and GUI development libraries.

Development Environment

- macOS (student machine)
- Python 3.x
- Virtual environment (optional)
- Tkinter (built-in)
- Jupyter Notebook (for model testing)

Libraries Used

Library	Purpose
Pandas	Data loading, cleaning, manipulation
NumPy	Numerical operations
Scikit-learn	TF-IDF vectorization, model training, logistic regression
Matplotlib	Visual charts and graphs
Tkinter	Graphical user interface
Joblib	Model saving and loading

Table 1: Libraries Used and Their Purposes

System Modules and Their Implementations

1. DataLoader Module

- Implemented in data_loader.py
- Uses Pandas to read CSV files
- Validates dataset format
- Returns a cleaned DataFrame to the GUI

2. SentimentModel Module

- Implemented in sentiment_model.py
- Loads pre-trained logistic regression model
- Uses TF-IDF vectorizer for transforming text
- Provides a predict() method to classify sentiment

3. MLModel Module

- Implemented in ml_model.py
- Handles:
    - Model training
    - Test/train splits
    - Accuracy evaluation
- Uses:
    - LogisticRegression
    - TfidfVectorizer

4. ProductAnalyzer Module

- Computes per-product statistics
- Counts positive and negative reviews
- Returns insights for GUI display

5. ChartBuilder Module

- Creates visual output using Matplotlib
- Generates:
    - Bar charts
    - Sentiment distribution charts

6. SentimentAppGUI Module

- Implemented in main.py
- Integrates all backend modules
- Provides:
    - File upload button
    - Sentiment analysis triggers
    - Chart display
    - Model training interface

System Workflow Summary

1. User selects a dataset
2. DataLoader loads and cleans the file
3. Text is preprocessed and vectorized
4. ML model predicts sentiment
5. Analyzer generates summaries
6. ChartBuilder creates visual graphs
7. GUI displays the final results

Code Structure (Folder Layout)

```
Final_project/
│
├── main_class.py
├── class_pkg.py/
    ├── __init__.py
    └── ai_recommend.py
    ├──analyzer.py
    └── sentiment.py
    ├── chart.py
    └── ML.py
    ├── dataL.py
    └── gui.py
```

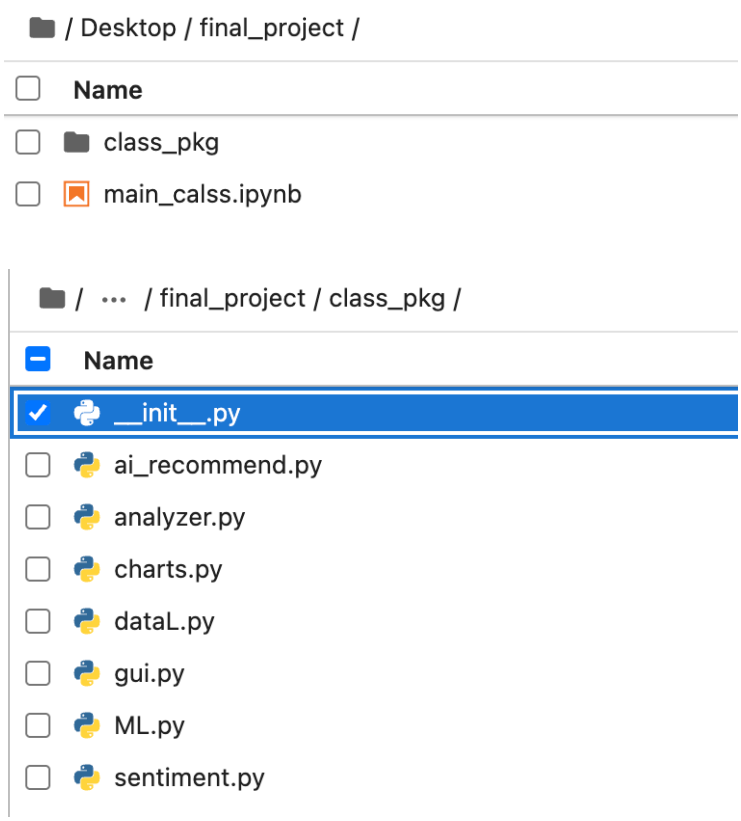

Figure 19: Project Directory Structure and Class Package Files

9.Testing

Testing was conducted to verify the correctness, reliability, and performance of the Sentiment Analyzer Dashboard. Multiple test cases were designed to ensure that the system behaves as expected across different components, including data loading, preprocessing, sentiment classification, visualization, and the graphical user interface.

12.1 Testing Objectives

The primary objectives of testing were:

1. To ensure that the system loads datasets correctly.
2. To verify that text preprocessing and TF-IDF vectorization function as intended.
3. To confirm that the machine learning model produces sentiment predictions without errors.
4. To check that product-level statistics are accurate.
5. To validate that charts and visualizations are generated properly.
6. To confirm that the GUI handles user interactions smoothly.

12.2 Test Environment

- Python 3.x
- macOS (developer machine)
- Tkinter GUI
- Sample CSV files from the Amazon reviews dataset

12.3 Test Cases

Test Case 1: Dataset Loading

Input: A valid CSV file
Expected Result:

- File loads successfully
- DataFrame appears in GUI

Status: Passed

Test Case 2: Data Loading with Invalid File

Input: TXT file or corrupted CSV
Expected Result:

- System displays an error message
- Prevents further processing

Status: Passed

Test Case 3: Sentiment Prediction

Input: Cleaned dataset with review text
Expected Result:

- System returns positive/negative labels
- No runtime errors occur

Status: Passed

Test Case 4: Model Training

Input: Training dataset
Expected Result:

- Model trains successfully
- Accuracy score is generated

Status: Passed

Test Case 5: Visualization Generation

Input: Summary sentiment data
Expected Result:

- Bar chart is generated and displayed
- Chart is readable with correct labels

Status: Passed

Test Case 6: GUI Interaction

Input: User clicks load, analyze, and chart buttons
Expected Result:

- All buttons trigger the correct functions
- Results appear correctly in the interface

Status: Passed

10.Conclusion

This project successfully developed a fully functional Sentiment Analyzer Dashboard capable of processing and evaluating large-scale Amazon review data. By integrating data preprocessing, TF-IDF vectorization, logistic regression classification, product-level analysis, and visualization into a unified GUI application, the system demonstrates a complete workflow for automated sentiment analysis. The results confirm that the model can effectively classify reviews and generate meaningful insights that support customers and vendors in understanding product satisfaction trends. The modular design of the system ensures maintainability and provides a strong foundation for future enhancements such as improved models, additional visualization options, and real-time data integration. Overall, the project meets its objectives and offers a practical demonstration of applying machine learning techniques to real-world text data.

References

1. Pedregosa, F., et al. *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825–2830, 2011.
2. Loper, E., & Bird, S. *Natural Language Processing with Python.* O'Reilly Media, 2009.
3. PromptCloud. *Amazon Unlocked Mobile Phone Review Dataset.*
4. McKinney, W. *Data Structures for Statistical Computing in Python.* Proceedings of the 9th Python in Science Conference, 2010.
5. Hunter, J. D. *Matplotlib: A 2D Graphics Environment.* Computing in Science & Engineering, 9(3), 90–95, 2007.
6. Tkinter Documentation — Python Software Foundation.
7. Logistic Regression Implementation — scikit-learn User Guide.

Appendix

Appendix A — Source Code Files

A.1 analyzer.py

```
 import pandas as pd

class ProductAnalyzer:

    def filter_product(self, df, keyword):
        if "Product Name" not in df.columns:
            print("ERROR: 'Product Name' column not found!")
            return pd.DataFrame()

        keyword = str(keyword).strip().lower()
        df["Product Name"] = df["Product Name"].astype(str).str.lower()

        filtered = df[df["Product Name"].str.contains(keyword, na=False)]
        return filtered

    def get_review_stats(self, df):
        if df.empty:
            return None

        lengths = df["Reviews"].astype(str).apply(len)
        return {
            "count": len(df),
            "columns": df.shape[1],
            "min_length": lengths.min(),
            "max_length": lengths.max(),
            "avg_length": lengths.mean()
        }

    def top_products(self, df, n=10):
        if df.empty:
            return None
        return df["Product Name"].value_counts().head(n)
```

A.2 ML.py

```
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import accuracy_score
import joblib

class MLModel:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(max_features=5000, stop_words="english")
        self.model = LogisticRegression(max_iter=200)

    def train(self, df, text_col="Reviews", label_col="Sentiment"):
        df = df[[text_col, label_col]].dropna()

        X = df[text_col].astype(str)
        y = df[label_col]

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        X_train_vec = self.vectorizer.fit_transform(X_train)
        X_test_vec = self.vectorizer.transform(X_test)

        self.model.fit(X_train_vec, y_train)

        preds = self.model.predict(X_test_vec)
        acc = accuracy_score(y_test, preds)

        return acc

    def predict(self, text):
        X = self.vectorizer.transform([str(text)])
        return self.model.predict(X)[0]

    def save(self, path="sentiment_model.pkl"):
        joblib.dump((self.model, self.vectorizer), path)

    def load(self, path="sentiment_model.pkl"):
        self.model, self.vectorizer = joblib.load(path)
```

A.3 charts.py

```
import matplotlib.pyplot as plt
import seaborn as sns

class ChartBuilder:

    def bar_chart(self, df):
```

```
        fig, ax = plt.subplots(figsize=(7, 4))
        sentiment_counts = df["Sentiment"].value_counts()
        ax.bar(sentiment_counts.index, sentiment_counts.values, color=["green", "blue",
"red"])
        ax.set_title("Sentiment Distribution (Bar)")
        ax.set_xlabel("Sentiment")
        ax.set_ylabel("Count")
        return fig

    def pie_chart(self, df):
        fig, ax = plt.subplots(figsize=(6, 6))
        sentiment_counts = df["Sentiment"].value_counts()
        ax.pie(sentiment_counts.values, labels=sentiment_counts.index,
autopct='%1.1f%%')
        ax.set_title("Sentiment Distribution (Pie)")
        return fig

    def histogram(self, df):
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.hist(df["Polarity"], bins=30, color="purple", alpha=0.7)
        ax.set_title("Polarity Histogram")
        ax.set_xlabel("Polarity")
        ax.set_ylabel("Frequency")
        return fig

    def line_chart(self, df):
        df2 = df.reset_index()
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.plot(df2.index, df2["Polarity"], color="blue")
        ax.set_title("Sentiment Polarity Over Index (Line Chart)")
        ax.set_xlabel("Review Index")
        ax.set_ylabel("Polarity")
        return fig

    def box_plot(self, df):
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.boxplot(df["Polarity"])
        ax.set_title("Polarity Distribution (Box Plot)")
        return fig

    def scatter_plot(self, df):
        fig, ax = plt.subplots(figsize=(7, 4))
        ax.scatter(df["Rating"], df["Polarity"], alpha=0.5, c="red")
        ax.set_title("Rating vs Polarity (Scatter Plot)")
        ax.set_xlabel("Rating")
        ax.set_ylabel("Polarity")
```

```
        return fig
```

A.5 ai_recommend.py

```
class AIRecommender:
    def generate(self, df):
        pos = len(df[df["Sentiment"] == "Positive"])
        neg = len(df[df["Sentiment"] == "Negative"])
        total = len(df)

        insights = []

        if total == 0:
            return ["No reviews available to generate insights."]

        if neg / total > 0.30:
            insights.append("A high percentage of negative reviews indicates potential
product issues or customer dissatisfaction.")

        if pos / total > 0.60:
            insights.append("The product has strong positive sentiment. Consider increasing
marketing efforts to boost sales.")

        if df["Polarity"].mean() < 0:
            insights.append("Average sentiment is negative. Customers are reporting
recurring issues.")

        if len(insights) == 0:
            insights.append("Overall sentiment is good and no major issues detected.")

        return insights
```

A.6 dataL.py

```
import pandas as pd
import zipfile

class DataLoader:

    def load_zip(self, path):
        with zipfile.ZipFile(path, 'r') as zf:
            csv_name = [f for f in zf.namelist() if f.endswith(".csv")][0]
            with zf.open(csv_name) as f:
                df = pd.read_csv(f)
        return df
```

```
def load_csv(self, path):
    return pd.read_csv(path)

def load_excel(self, path):
    return pd.read_excel(path)
```

Appendix B — Dataset Sample

A	B	C	D	E	F	G
Product Nam	Brand Name	Price	Rating	Reviews	Review Votes	
"CLEAR CLEA	Samsung	199.99	5	I feel so LUCKY to have found this used (phone to us & not used hard at	1	
"CLEAR CLEA	Samsung	199.99	4	nice phone, nice up grade from my pantach revue. Very clean set up ar	0	
"CLEAR CLEA	Samsung	199.99	5	Very pleased	0	
"CLEAR CLEA	Samsung	199.99	4	It works good but it goes slow sometimes but its a very good phone I lov	0	
"CLEAR CLEA	Samsung	199.99	4	Great phone to replace my lost phone. The only thing is the volume up	0	
"CLEAR CLEA	Samsung	199.99	1	I already had a phone with problems... I know it stated it was used, but	1	
"CLEAR CLEA	Samsung	199.99	2	The charging port was loose. I got that soldered in. Then needed a new	0	
"CLEAR CLEA	Samsung	199.99	2	Phone looks good but wouldn't stay charged, had to buy new battery. S	0	
"CLEAR CLEA	Samsung	199.99	5	I originally was using the Samsung S2 Galaxy for Sprint and wanted to r	0	
"CLEAR CLEA	Samsung	199.99	3	It's battery life is great. It's very responsive to touch. The only issue is t	0	
"CLEAR CLEA	Samsung	199.99	3	My fiance had this phone previously, but caused many problems. So, o	0	
"CLEAR CLEA	Samsung	199.99	5	This is a great product it came after two days of ordering it. There was	0	
"CLEAR CLEA	Samsung	199.99	5	These guys are the best! I had a little situation with my item but they q	2	
"CLEAR CLEA	Samsung	199.99	1	I'm really disappointed about my phone and service. The phone went o	1	
"CLEAR CLEA	Samsung	199.99	5	Ordered this phone as a replacement for the same model until my con	1	
"CLEAR CLEA	Samsung	199.99	2	Had this phone before and loved it but was not working so I got this pho	0	
"CLEAR CLEA	Samsung	199.99	5	I was able to get the phone I previously owned...with a keyboard and to	6	
"CLEAR CLEA	Samsung	199.99	5	I brought this phone as a replacement for my daughter, who is very har	0	
"CLEAR CLEA	Samsung	199.99	4	I love the phone. It does everything I need and works great. I purchased	1	
"CLEAR CLEA	Samsung	199.99	3	unfortunately Sprint could not activate the phone due to the blocking	0	
"CLEAR CLEA	Samsung	199.99	4	The battery was old & had been over used because it barely holds a ch	0	
"CLEAR CLEA	Samsung	199.99	4	pros-beautiful screen,capable of running chrome,take good pictures,t	0	
"CLEAR CLEA	Samsung	199.99	1	I purchased this phone in December as a christmas present to my son	19	
"CLEAR CLEA	Samsung	199.99	4	Phone good just a little slow phone old but it's a great phone temporar	0	
"CLEAR CLEA	Samsung	199.99	4	Phone's speaker little low. Overall very happy with the phone. I would p	0	
"CLEAR CLEA	Samsung	199.99	5	the phone was great and in good condition. My Daughter is so happy th	0	
"CLEAR CLEA	Samsung	199.99	3	the reasons for the 3 star rating was it was in my opinion better than m	0	
"CLEAR CLEA	Samsung	199.99	5	Phone works great. No problems at all	0	
"CLEAR CLEA	Samsung	199.99	1	was not in good condition but does work good	0	
"CLEAR CLEA	Samsung	199.99	1	Just... not good. The phone has great screen resolution, storage is low,	0	
"CLEAR CLEA	Samsung	199.99	5	as described, fast ship!	0	
"CLEAR CLEA	Samsung	199.99	5	Perfect in every way.	0	
"CLEAR CLEA	Samsung	199.99	2	One of the phones have a bad charger port. I want to send it back	0	